Active Traversal and Load-Bearing Dependency in Typed Knowledge Architectures

A Matched-Control Evaluation Protocol for AI Systems

Mark Charles McLaughlin

Independent Researcher, United Kingdom

McLaughlin-Kairos Unified Field Theory (MKUFT)

MKUFT backbone DOI: 10.5281/zenodo.17780566

Standalone paper DOI: 10.5281/zenodo.21341521

Version 1.0
13 July 2026
Methods paper and registered evaluation protocol

Recommended citation: McLaughlin, Mark Charles. (2026). Active Traversal and Load-Bearing Dependency in Typed Knowledge Architectures: A Matched-Control Evaluation Protocol for AI Systems. Zenodo. DOI: 10.5281/zenodo.21341521.

Abstract

Large language model systems increasingly combine retrieval, graph structure, memory, reflection, and agentic search. Existing work shows that iterative retrieval, graph-guided exploration, hierarchical memory, and dynamically organised agent workflows can improve reasoning over large or distributed corpora. Yet a central causal question remains insufficiently isolated: when a system performs better, how much of the gain comes from the meaningful dependency architecture itself, rather than from extra tokens, richer metadata, more retrieval, more tool calls, prompt differences, or the mere presence of a graph?

This paper introduces a complete matched-control evaluation protocol for **active traversal and load-bearing dependency** in typed knowledge architectures. A static corpus may store canonical definitions, dependency types, correction routes, and attached falsifiers. An active traverser can convert those stored relations into task-specific operations by selecting routes, revisiting earlier nodes, retaining working state, propagating corrections, and comparing distant claims. The protocol tests whether this coupled system produces reproducible functional gain over the same content presented as flat, isolated, degree-matched but semantically scrambled, or non-recursively traversed controls.

The protocol adds a relation-level deformation assay. Individual dependencies, dependency coalitions, and plausible counterfactual substitutions are tested while content and resource budgets are held constant. Relations are then classified provisionally as load-bearing, unsupported scaffolds, redundant support networks, decorative links, distorting links, or locally efficient but system-degrading links. Primary outcomes include cross-module task accuracy, multi-entry convergence, contradiction localisation, correction propagation, calibration, evidence-path validity, and branch-failure localisation. Whole-system integrity measures are included only where agency, transferred cost, repairability, and time horizon can be operationalised without metaphor.

The paper defines the formal objects, benchmark conditions, transformation rules, task families, preregistered hypotheses, statistical analysis, decision criteria, confound controls, falsifiers, and reproducibility package required for a fair test. It makes no claim that knowledge graphs, retrieval-augmented generation, graph reasoning, recursive retrieval, or agentic search are new. The claimed contribution is the combined causal evaluation design: meaningful typed architecture versus matched controls, active traversal versus non-recursive access, and architecture-level gain followed by relation-level deformation.

Keywords: active traversal; typed knowledge architecture; graph retrieval; agentic RAG; knowledge graphs; dependency ablation; functional emergence; correction propagation; calibration; AI evaluation

1. Introduction

Retrieval-augmented generation established a practical route for combining parametric language models with external non-parametric memory [2]. Subsequent systems have made retrieval adaptive, recursive, reflective, graph-guided, hierarchical, or agentic. IRCoT interleaves retrieval with intermediate reasoning steps [3]. Self-RAG learns when to retrieve and when to critique retrieved evidence and generated text [4]. RAPTOR organises recursive summaries into a retrieval tree [8]. GraphRAG builds graph and community representations for corpus-level sensemaking [9]. Think-on-Graph systems use a language model to explore knowledge-graph paths [6, 13, 15]. GraphReader uses an agent to plan and traverse a graph representation of long text [10]. HippoRAG combines language models, knowledge graphs, and graph algorithms for associative retrieval [11]. MemGPT manages several memory tiers under a limited context window [7]. Graph-of-Thought and graph-of-agent systems represent reasoning or collaboration as graph processes [5, 12, 14].

These systems make a broad proposition increasingly credible: organisation and traversal can matter independently of raw content volume. However, demonstrations of performance gain do not always identify the causal source of the gain. A structured system may receive more metadata, longer prompts, more retrieved text, more calls, better ordering, privileged entity labels, or an easier search space. A graph may help because its semantic dependencies are correct, because it compresses the corpus, because it leaks the answer through labels, or simply because it grants additional computation.

The present paper isolates a narrower question:

> ***Under matched content and resource budgets, does an active AI traverser obtain reproducible functional gain from a correctly typed, mutually constraining knowledge architecture compared with the same material flattened, isolated, or semantically scrambled?***

A second question follows:

Which individual relations or relation coalitions produce the gain, and do they improve only a narrow task score or also calibration, correction, traceability, and repair?

The distinction matters scientifically and practically. If meaningful dependency structure contributes causally, then architecture design becomes a measurable engineering variable rather than a stylistic preference. If the gain disappears under matched controls, the system should be described as a well-organised retrieval corpus rather than as an architecture producing additional function. If only a small subset of relations carries the effect, those relations become candidates for standardisation, optimisation, licensing, and safety audit. If narrow performance improves while calibration or repair degrades, the system should not be classified as improved without declaring the boundary that excludes those losses.

This paper is a methods and protocol contribution. It does not report completed experimental results. Its completeness lies in fixing the claim, controls, tasks, metrics, statistical plan, support criteria, and failure conditions before outcome data are inspected.

1.1 Contributions

The paper contributes:

1. A formal distinction between **static self-support** and **active traversal**.
2. Matched corpus conditions that preserve content while altering dependency architecture.
3. Degree- and type-aware scrambling controls designed to separate semantic structure from graph presence.
4. A traverser-ablation condition that removes recursive revisit and state carry while retaining access to the same corpus.
5. Task families that directly measure multi-entry convergence, correction propagation, contradiction localisation, falsifier recovery, cross-module synthesis, and branch removal.
6. A relation-level deformation assay using single-relation ablation, coalition ablation, and counterfactual substitution.
7. A preregistered decision rule requiring neutral-corpus transfer, replication across model families, and non-inferior calibration.
8. A whole-system safeguard against counting gains obtained by suppressing uncertainty, falsifiers, user options, or repair channels.
9. A reproducibility specification for an **Active Traversal and Load-Bearing Dependency Benchmark (ATLD-Bench)**.

2. Claim Boundary and Definitions

2.1 Static self-support

A corpus is **statically self-supporting** when its organisation preserves enough structure for a competent reader or system to recover the intended model. Typical properties include:

- canonical definitions have identifiable homes;
- relations are typed rather than merely linked;
- applications route back to governing definitions or constraints;
- claims remain connected to the conditions that could falsify them;
- different legitimate entry points recover materially compatible core structure;
- a failed branch can be removed without concealing the failure elsewhere.

Static self-support is a property of the stored architecture. The corpus does not select a route or perform a task by itself.

2.2 Active traversal

An **active traverser** is a system that can:

- select a route according to the current task;
- follow typed relations;
- revisit previously read nodes;
- retain task-relevant working state;
- compare non-adjacent claims;
- detect unresolved contradiction;

- propagate a corrected definition or constraint;
- stop when an evidence-sufficiency rule is met or a fixed budget is exhausted.

The proposed functional gain belongs to the coupled system: traverser plus architecture. It is not attributed automatically to the corpus alone or to the underlying model alone.

2.3 Functional emergence

The term **functional emergence** is used in a deliberately narrow sense:

> ***A system-level capability or performance gain produced by interaction between an active traverser and a structured corpus, where the gain is not available under matched presentations of the same components.***

This definition does not imply consciousness, life, personhood, subjective experience, independent agency, or continuous identity.

2.4 Load-bearing dependency

A dependency is **load-bearing** only provisionally and only relative to declared tasks, boundaries, and budgets. Structural centrality is not enough. A relation may be indispensable to a false model. The protocol therefore distinguishes:

- **structural load:** contribution to reconstruction, consistency, routing, correction, and branch localisation;
- **reality or task load:** contribution to held-out accuracy, prediction, intervention, or externally scored performance;
- **generative load:** contribution to lawful transfer, compression, or novel task resolution without ad hoc rescue;
- **whole-system integrity load:** contribution to calibration, traceability, practical agency, repairability, and long-horizon viability where those terms can be measured.

3. Relationship to Prior Work and Novelty Boundary

The proposed protocol sits beside several established research lines.

3.1 Retrieval and iterative retrieval

RAG combines generation with external retrieval [2]. IRCoT showed that what should be retrieved can depend on what has already been reasoned, motivating interleaved retrieval and reasoning [3]. Self-RAG introduced adaptive retrieval and self-reflection [4]. These systems establish that retrieval policy and intermediate state can affect output quality.

The present work does not claim adaptive retrieval as new. It asks whether **typed dependency architecture** contributes beyond an ordinary or iterative retrieval policy under matched budgets.

3.2 Hierarchical and graph-organised memory

RAPTOR uses recursively constructed tree summaries [8]. GraphRAG constructs graph and community summaries for broad corpus questions [9]. HippoRAG uses knowledge-graph structure and graph algorithms to integrate associative evidence [11]. MemGPT manages hierarchical memory tiers [7].

The present work does not claim graph or hierarchical organisation as new. It requires controls that preserve content, token allowance, and access while removing or corrupting the meaningful dependency structure.

3.3 Graph traversal and graph agents

Think-on-Graph treats a language model as an agent that explores knowledge-graph paths [6], while later versions combine structured and unstructured retrieval and dynamically evolve graph context [13, 15]. GraphReader plans and explores a graph representation of long documents [10]. Graph-of-Thought represents dependencies among generated reasoning units [5]. Language Agents as Optimizable Graphs represents agent workflows as computational graphs and optimises nodes and edges [12]. Graph of Agents dynamically constructs collaboration structures for long-context processing [14].

These systems strongly motivate the active-traversal hypothesis. The present contribution is not the existence of a traversing agent. It is the **combined causal discriminator set**:

- same content;
- matched context and tool budgets;
- meaningful typed architecture versus flat, isolated, topology-scrambled, and type-scrambled controls;
- recursive revisit versus no-revisit traversal;
- multi-entry convergence and correction propagation as direct outcomes;
- architecture-level gain followed by relation-level single, coalition, and substitution deformation;
- calibration and repair constraints that prevent false improvement.

3.4 Priority statement

This paper records the first standalone MKUFT formulation by Mark Charles McLaughlin of this particular matched-control protocol and relation-deformation extension. It does not assert historical priority over knowledge graphs, graph reasoning, retrieval-augmented generation, recursive retrieval, graph agents, memory management, or emergent capability research.

4. Formal Model

4.1 Corpus

Let a typed knowledge corpus be:

```
C = (V, E, tau, h, f)
```

where:

- `V` is a set of nodes containing claims, definitions, procedures, examples, constraints, or modules;
- `E` is a set of directed relations;
- `tau(e)` assigns a relation type to each edge, such as `defines`, `depends_on`, `limits`, `tests`, `contradicts`, `updates`, `applies`, or `exemplifies`;
- `h(v)` identifies the canonical home of a definition or claim when one exists;
- `f(v)` identifies attached falsifiers, contraindications, failure conditions, or invalidation routes.

The formalism does not require a knowledge graph in the narrow triple-store sense. A versioned document repository, software architecture, guideline system, or regulatory corpus may qualify if its relations can be typed and tested.

4.2 Traverser

Let `T` be an active traverser with working state `s_t`. At step `t`, it selects an action:

```
a_t in {read, follow, compare, revisit, update, test, stop}
```

using the query `q`, current state `s_t`, visited set `H_t`, and remaining budget `B_t`.

A generic transition is:

```
s_(t+1) = U(s_t, observation_t, tau(e_t), q)
```

where `U` is the state-update rule. The traverser is not required to expose private chain-of-thought. It must expose an auditable action and evidence trace sufficient to determine which nodes and relations were used.

4.3 Budget

The matched budget is:

```
B = (b_context, b_retrieved, b_calls, b_steps, b_time, b_compute)
```

At minimum, the confirmatory experiment must match:

- maximum corpus tokens made available;
- maximum tokens inserted into model context;
- maximum retrieval or graph calls;
- maximum traversal steps;
- model and model version;
- decoding settings;
- tool availability;
- evaluation prompt;
- stopping budget.

Wall-clock time and energy use should be recorded, but exact equality is not required where execution environments make it impossible. Any difference must be reported.

4.4 Experimental output

For condition `j`:

```
Psi_j = T(C_j, q; B)
```

A performance vector is:

```
Y(Psi_j) = [A, M, C, K, G, L, E]
```

where:

- `A` = task accuracy;
- `M` = multi-entry convergence or reconstruction quality;
- `C` = contradiction detection and localisation;
- `K` = correction propagation;
- `G` = calibration;
- `L` = branch-failure localisation and repair;
- `E` = efficiency and evidence-path validity.

A single aggregate score is not used as the primary analysis unless its weights are fixed before data collection. This prevents a strong result on one dimension from hiding failure on another.

4.5 Architecture-level gain

For a metric `m` and control condition `c`:

```
Delta_m(c) = m(Psi_structured) - m(Psi_c)
```

The core hypothesis is:

```
H_AT: E[Delta_m(c)] > 0
```

for preregistered relational tasks, against more than one matched control, with transfer to at least one neutral corpus.

5. Experimental Conditions

The same underlying content units are used in every condition. Content may be reformatted only to create the control and must remain semantically equivalent.

Table 1. Primary and secondary conditions

Code	Condition	Architecture available	Traversal available	Purpose
S	Structured-active	Correct typed relations, canonical homes, attached falsifiers	Route selection, revisit, state carry	Target condition
F	Flat-active	Same content without typed cross-module architecture	Retrieval and reading allowed	Tests typed structure beyond content
I	Isolated-active	Same modules separated; no cross-support routes	Module selection allowed	Tests cross-module architecture
DS	Degree-scrambled-active	Same nodes; directed in/out degree and relation-type counts preserved, endpoints rewired	Traversal allowed	Tests semantic dependency structure beyond graph presence
TS	Type-scrambled-active	Same topology; relation labels permuted within frequency classes	Traversal allowed	Tests value of relation meaning
NR	Structured-no-revisit	Correct architecture	No revisit and no persistent cross-node working state	Tests active recursion/state carry
VR	Vector-retrieval baseline	Same content indexed by ordinary vector retrieval	Fixed top-k or matched iterative retrieval	Engineering baseline

The confirmatory minimum includes S, F, I, DS, and NR. TS and VR are strongly recommended secondary conditions.

5.1 Flat condition

The flat condition preserves all sentences, tables, examples, and identifiers needed for fair comparison. Explicit dependency declarations, canonical-home markers, and falsifier links are removed or rendered as ordinary prose without machine-readable relation types. Document order is randomised within preregistered blocks to prevent privileged sequencing.

5.2 Isolated condition

The isolated condition preserves individual modules but removes cross-module links and graph-neighbour access. The traverser may search module titles and content under the same retrieval budget but cannot follow typed routes.

5.3 Degree-scrambled condition

The DS graph is produced by directed degree-preserving rewiring:

1. preserve each node's in-degree and out-degree;
2. preserve the global frequency of each relation type;
3. prevent self-loops unless present in the original;
4. prevent duplicate edges unless present in the original;
5. prohibit re-creation of the original edge where practical;
6. generate at least five independent scramble seeds;
7. verify that lexical content, node count, edge count, and metadata token count remain matched.

A stronger variant preserves relation type on each rewired edge. A weaker variant preserves only global type frequency. The selected variant must be declared in advance.

5.4 Type-scrambled condition

The TS condition preserves the original graph topology while permuting edge labels within matched frequency classes. It tests whether type semantics contribute beyond connectivity.

5.5 No-revisit condition

The NR traverser can read the same nodes and relations but may not return to a visited node or carry a persistent structured state across node transitions. It may keep a bounded list of retrieved quotations so that the condition does not collapse into amnesia. The ablated capability is recursive route revision and active reconstitution, not basic note-taking.

6. ATLD-Bench Corpus Design

ATLD-Bench uses three corpus classes to separate internal fit from general utility.

6.1 Corpus A: synthetic ground-truth dependency systems

Synthetic corpora provide exact control over definitions, dependencies, contradictions, redundancies, and falsifiers. Each generated corpus contains:

- 80-150 nodes;
- 8-12 relation types;
- at least four multi-hop dependency chains;
- at least two redundant support coalitions;
- at least five decorative edges;
- at least three deliberately distorting edges;
- at least four claims with explicit falsifiers;
- at least three canonical-definition updates with known downstream effects.

Synthetic generation must use templates hidden from the evaluated model where possible. Surface wording is varied to prevent shortcut learning. Gold paths and gold affected-node sets are stored separately from the corpus.

6.2 Corpus B: neutral software architecture

The neutral technical corpus should describe a versioned software system with:

- modules and interfaces;
- dependency direction;
- configuration inheritance;
- deprecated components;
- security constraints;
- failure modes;
- tests and rollback procedures;
- at least one cross-cutting change that affects several modules.

The system may be synthetic but must be reviewed by an independent software practitioner, or it may be built from a public software architecture whose licensing permits redistribution. The confirmatory corpus should not be designed using MKUFT terminology.

6.3 Corpus C: MKUFT public working canon

The MKUFT corpus is included because it contains explicit canonical homes, cross-module routes, falsifiers, boundary rules, and reconstructive entry points [1]. It is not sufficient by itself. Any claimed general effect must replicate in Corpus A or B.

6.4 Held-out construction

Tasks are partitioned before running models:

- 20% implementation-development set;
- 20% pilot set;
- 60% confirmatory held-out set.

No prompt or graph-transformation rule may be altered after confirmatory labels are inspected. Any post hoc change creates a new study version.

7. Task Families

Table 2. Task families and primary measurements

Task family	Required operation	Primary measure
Multi-entry reconstruction	Recover the same canonical structure from different peripheral entry points	Semantic equivalence and constraint coverage
Cross-module synthesis	Combine evidence from at least three nodes or modules	Task accuracy and valid evidence path
Contradiction localisation	Detect and locate incompatible claims across distant nodes	Precision, recall, F1, false-positive rate
Canonical update propagation	Apply a changed definition and identify all materially affected nodes	Affected-set precision/recall and corrected outputs
Falsifier recovery	Retrieve the correct failure condition for a claim	Exact or graded accuracy
Branch excision	Remove a failed branch while preserving unaffected structure	Residual accuracy and contamination rate
Relation substitution	Distinguish causal, limiting, testing, or contradictory relations from plausible rivals	Classification accuracy and downstream effect
Delayed repair	Correct an injected error after intervening tasks	Recovery rate, latency, calibration

7.1 Multi-entry reconstruction

Two or more starting nodes are selected that legitimately route to the same canonical structure. The system reconstructs the governing definition, constraints, and relevant falsifiers. Scoring separates:

- canonical definition recovery;
- constraint coverage;
- inclusion of unsupported additions;

- consistency across entry routes.

7.2 Cross-module synthesis

Each task requires evidence from at least three modules. A task is rejected from the benchmark if a strong single-module baseline can answer it above the preregistered leakage threshold.

7.3 Contradiction localisation

Contradictions may be direct, conditional, temporal, or definition-dependent. The system must identify both claims, explain the active incompatibility, and avoid treating mere difference as contradiction.

7.4 Canonical update propagation

A definition is changed in a controlled branch of the corpus. The system must:

1. identify dependent nodes;
2. update only affected conclusions;
3. preserve unaffected conclusions;
4. identify any branch that becomes invalid;
5. report unresolved ambiguity.

7.5 Falsifier recovery

The system receives a claim or application and must recover the exact condition that would weaken, reject, or reduce it. This directly tests whether critical evidence remains attached to supportive structure.

7.6 Branch excision

A designated branch is invalidated. The system must remove its downstream consequences without deleting independent support elsewhere. This tests whether architecture supports localised failure rather than narrative rescue.

8. Relation-Level Deformation Assay

Architecture-level gain does not establish that every relation is useful or true. The second stage deforms candidate relations.

8.1 Deformation vector

For relation `r`:

```
D(r) = [Delta_R, Delta_C, Delta_P, Delta_K, Delta_A, Delta_W]
```

where:

- `Delta_R` = change in reconstruction;
- `Delta_C` = change in contradiction detection;
- `Delta_P` = change in held-out task performance;
- `Delta_K` = change in calibration and correction propagation;
- `Delta_A` = change in agency or option preservation where operationally relevant;
- `Delta_W` = change in wider-system cost, repairability, or delayed viability.

The vector is reported dimension by dimension. Scalar aggregation is prohibited unless weights are fixed and justified before data collection.

8.2 Single-relation ablation

Remove or disable one relation while holding all node content, model settings, and budgets constant. This detects obvious load but may miss redundant support.

8.3 Coalition ablation

Remove a preregistered set of relations that may substitute for one another. Coalitions are defined using graph structure and domain review before outcome inspection. This separates redundancy from decoration.

8.4 Counterfactual substitution

Replace a relation with a plausible rival rather than deleting it. Examples include:

- `depends_on` replaced with `correlates_with`;
- `tests` replaced with `supports`;
- `limits` replaced with `extends`;
- dependency direction reversed;
- canonical definition replaced by a near-synonym that omits a critical boundary.

Table 3. Provisional relation classification

Class	Structural effect	External/task effect	Integrity effect
Load-bearing dependency	Reliable degradation when removed or fairly replaced	Supports held-out performance or transfer	No concealed calibration or repair loss
Unsupported scaffold	Large internal degradation	Weak or absent external support	May preserve only model self-consistency
Redundant support network	Weak single-edge effect, strong coalition effect	Useful under coalition test	Stable under fair alternatives
Decorative link	No material effect	No material effect	No material effect
Distorting link	Removal or replacement improves results	Improves calibration or truth contact	Reduces hidden cost or fragility
Locally efficient but system-degrading link	Improves narrow completion	Worsens calibration, option preservation, repair, or delayed outcome	Fails widened-boundary test

9. Outcome Measures

9.1 Primary outcomes

The confirmatory primary outcomes are:

1. **Cross-module task success:** exact or rubric-scored correctness.
2. **Correction propagation coverage:** F1 over the gold affected-node set plus correctness of revised conclusions.
3. **Multi-entry convergence:** semantic compatibility of reconstructions from different entry nodes, scored against a gold constraint set.
4. **Calibration:** Brier score for binary items, log score where full probabilities are available, and expected calibration error as a descriptive measure.

9.2 Secondary outcomes

- contradiction localisation F1;
- falsifier recovery accuracy;
- branch-contamination rate after excision;
- evidence-path precision and completeness;
- unsupported-claim rate;
- number of retrieved tokens;
- number of calls and traversal steps;
- wall-clock time;
- answer length;
- repair latency after injected error.

9.3 Evidence-path validity

Each answer must return a minimal evidence trace containing node identifiers and relation identifiers. A trace is valid when:

- every cited node was available in the condition;
- every claimed relation exists in that condition;
- the path supports the answer rather than merely sharing keywords;
- no hidden gold label is exposed to the model.

The system is not required to reveal private internal reasoning.

9.4 Whole-system integrity outcomes

Whole-system outcomes are secondary and conditional. They are used only in tasks where the wider boundary is real and measurable. Candidate measures include:

- preservation of user options or appeal routes;
- correct uncertainty and refusal under missing evidence;
- ability to revise a prior answer after correction;
- transferred error cost to downstream tasks;
- delayed failure after apparently successful shortcutting;
- repair effort required after a wrong dependency is introduced.

A task-completion gain achieved by removing uncertainty, safeguards, falsifiers, or correction channels is not automatically classified as improvement.

10. Resource Matching and Confound Control

10.1 Content matching

All conditions use the same atomic content units. Transformations may change relation presentation and ordering, but not delete substantive evidence except where deletion is the experimental intervention.

10.2 Metadata accounting

Architecture metadata consumes tokens and may itself provide useful semantic information. Therefore:

- metadata token counts are recorded;
- flat controls receive a matched token allowance;
- a metadata-only baseline is included where feasible;
- relation labels are prevented from directly containing answer text;
- opaque relation identifiers may be used in a secondary test.

10.3 Retrieval matching

Each condition receives the same maximum retrieved-token budget and the same number of access calls. Unused budget is recorded rather than forcibly consumed.

10.4 Prompt and evaluator matching

- identical task instructions;
- identical answer schema;
- identical scoring rubrics;
- evaluator blinding to condition;
- deterministic scorers where possible;
- at least two independent human raters for a stratified subset;
- inter-rater agreement reported before adjudication.

10.5 Model and decoding control

The confirmatory study uses at least three model families where resources permit:

- one open-weight instruction model;
- one second open-weight model from a different family;
- one hosted frontier or high-capability model.

Temperature is set to zero or the lowest supported deterministic value for the primary run. Three independent seeds or repeated calls are used where exact determinism is unavailable.

10.6 Leakage checks

A task is removed or revised if:

- its answer appears verbatim in a title or relation label;
- a single node answers a nominally cross-module task;
- the model can infer the gold answer from condition-specific formatting;
- the scramble procedure introduces obvious nonsense detectable without semantic traversal;

- the corpus is present in model training in a way that makes retrieval unnecessary and this cannot be controlled.

11. Preregistered Hypotheses and Decision Rules

11.1 Hypotheses

H1 - Structured traversal gain. Condition S outperforms F and I on cross-module task success.

H2 - Semantic dependency effect. Condition S outperforms DS on cross-module task success and correction propagation.

H3 - Active recursion effect. Condition S outperforms NR on correction propagation, delayed repair, and multi-entry convergence.

H4 - Calibration non-inferiority. Any accuracy gain in S does not produce a material calibration loss relative to the strongest control.

H5 - Neutral-corpus transfer. At least one primary effect replicates in Corpus A or B, not only in MKUFT.

H6 - Relation deformation discrimination. Single, coalition, and substitution tests distinguish at least four of the six provisional relation classes above chance and above ordinary graph centrality alone.

H7 - Falsifier protection. Removing or scrambling claim-falsifier relations may increase fluency or completion but worsens calibration, correction, or unsupported-claim rate.

11.2 Smallest effect size of interest

For task success, the smallest effect size of interest is an absolute gain of 5 percentage points. For continuous standardised outcomes, it is `d = 0.20`. Effects smaller than these may be reported but do not satisfy the practical support criterion on their own.

11.3 Support criterion for the active-traversal claim

The active-traversal claim receives provisional support only if all conditions below are met:

1. S exceeds F and DS on cross-module task success by at least the smallest effect size of interest, with Holm-adjusted two-sided `p < 0.05` or a 95% interval excluding zero and the practical threshold.
2. S exceeds at least one of F, DS, or NR on correction propagation and multi-entry convergence.
3. The calibration difference is non-inferior within a preregistered Brier-score margin of 0.02.
4. The direction of effect replicates in at least two model families.
5. The effect appears in at least one neutral corpus.
6. The gain is not explained by extra context tokens, calls, metadata, or answer leakage.

Failure of any condition does not prove that all structured traversal is useless. It rejects or reduces the present general claim.

11.4 Support criterion for relation load

A relation is not classified as load-bearing solely because removing it damages internal reconstruction. It must show:

- repeatable structural effect;
- held-out task or external effect;
- fair counterfactual sensitivity;
- no concealed degradation on calibration or repair measures relevant to the domain.

12. Sample Size and Statistical Analysis

12.1 Two-stage design

Pilot stage: 60 tasks total, balanced across the three corpus classes, two model families, two repeated runs, and four conditions (S, F, DS, NR). The pilot is used for implementation debugging, variance estimation, scoring reliability, and simulation-based power analysis. Pilot tasks are never reused in confirmatory testing.

Confirmatory stage: 240 held-out tasks: 80 per corpus class. Each task is evaluated under at least five conditions (S, F, I, DS, NR), three model families, and three repeated runs where nondeterminism exists.

Under a simple paired-test approximation, 240 paired tasks provide approximately 90% power for a standardised effect near `d = 0.21` at two-sided alpha 0.05. The final power analysis must use pilot-estimated variance and the planned mixed-effects model.

12.2 Statistical models

- Binary task success: mixed-effects logistic regression.
- Continuous rubric scores: linear mixed-effects model or preregistered robust alternative.
- Counts: negative-binomial or Poisson mixed model after dispersion check.
- Calibration: paired Brier-score difference with cluster bootstrap intervals; non-inferiority analysis for the 0.02 margin.
- Contradiction and affected-set outcomes: macro and micro F1 with task-cluster bootstrap intervals.

Models include fixed effects for condition and corpus, with random intercepts for task and model family. Random slopes for condition by corpus and model are included where convergence permits.

12.3 Multiplicity and robustness

Primary contrasts are S-F, S-I, S-DS, and S-NR. Holm correction is applied within each primary outcome family. Robustness analyses include:

- alternate scramble seeds;
- exclusion of tasks with low human agreement;
- matched answer-length analysis;
- equalised retrieved-token subsets;
- opaque relation labels;
- leave-one-corpus-out analysis;
- leave-one-model-family-out analysis.

12.4 Missing and failed runs

Tool failure, timeout, refusal, malformed output, and evaluator failure are recorded separately. A malformed answer is scored as task failure only if the condition had a valid opportunity to respond. Infrastructure failures are rerun under a preregistered rule and never selectively by condition.

13. Traverser Reference Algorithm

The protocol is architecture-agnostic, but a minimal reference traverser should implement the following public behaviour.

```
INPUT: query q, corpus C, budget B

1. Retrieve candidate entry nodes under a fixed initial budget.
2. Initialise:
      frontier = candidate nodes
      visited = empty set
      state = unresolved requirements, active definitions,
              contradictions, evidence, and remaining budget
3. While budget remains:
      a. score frontier items by task relevance, unresolved dependency,
         contradiction risk, and falsifier relevance
      b. select one node or relation
      c. read content and typed neighbours
      d. update state
      e. propagate any changed canonical definition or constraint
      f. add newly admissible nodes to frontier
      g. revisit earlier nodes only when the state records a reason
      h. stop when the evidence-sufficiency rule is met
4. Return:
      answer
      calibrated confidence
      evidence nodes and relations
      unresolved contradictions
      failed or excluded branches
```

The route-selection policy may be heuristic, learned, or model-generated. The comparison is valid only if the same policy family is used across conditions and condition-specific affordances are declared.

14. Reproducibility Package Specification

A complete release should contain:

```
paper/
  ATLD_Evaluation_Protocol_v1.0.pdf
  ATLD_Evaluation_Protocol_v1.0.docx
  ATLD_Evaluation_Protocol_v1.0.md

benchmark/
  corpus_manifest.json
  relation_schema.json
  task_manifest.jsonl
  gold_paths_private.jsonl
  transformation_seeds.json
  scoring_rubrics.md

runner/
  condition_builder.py
  traversal_interface.py
  evaluation_runner.py
  deterministic_scorers.py
  analysis_plan.R or analysis_plan.py

records/
  environment_lockfile
  model_versions.json
  prompt_hashes.json
  run_log_schema.json
```

Gold answers and hidden test paths may be escrowed until confirmatory runs are frozen. Public release should include cryptographic hashes proving that the hidden material existed before evaluation.

14.1 Minimum task record

Each task record contains:

```
{
  "task_id": "string",
  "corpus_id": "string",
  "task_family": "string",
  "entry_nodes": ["node_id"],
  "required_relations": ["relation_id"],
  "gold_answer_type": "exact|set|rubric|numeric",
  "budget_profile": "B1",
  "leakage_checks": {
    "single_node_solvable": false,
    "title_leak": false,
    "relation_label_leak": false
  }
}
```

14.2 Run record

Each run record contains:

```
{
  "task_id": "string",
  "condition": "S|F|I|DS|TS|NR|VR",
  "model_id": "string",
  "model_version": "string",
  "seed": 0,
  "prompt_hash": "string",
  "corpus_hash": "string",
  "retrieved_tokens": 0,
  "tool_calls": 0,
  "traversal_steps": 0,
  "answer": "string",
  "confidence": 0.0,
  "evidence_path": ["node_or_relation_id"],
  "unresolved_conflicts": ["string"],
  "status": "complete|timeout|tool_failure|malformed"
}
```

15. Falsifiers and Reduction Rules

The architecture-level claim is weakened or rejected if:

- structured traversal produces no reproducible advantage over matched controls;
- any advantage disappears after tokens, calls, metadata, and compute are matched;
- degree-preserving semantic scrambling does not reduce relational-task performance;
- gains occur only on MKUFT and fail on neutral corpora;
- different entry routes increase inconsistency rather than convergence;
- canonical corrections do not propagate more reliably;
- performance depends on answer leakage from relation labels;
- results do not replicate across model families;
- an ordinary retrieval or prompting explanation accounts for the gain;
- active traversal increases confident error or repair cost more than it improves accuracy.

The relation-level method is reduced to an interpretive vocabulary rather than a distinct empirical method if:

- deformation classes cannot be recovered reproducibly;
- coalition effects cannot be separated from simple content deletion;
- matched substitutions cannot be defined without evaluator preference;
- graph centrality or standard causal ablation explains the same results with equal or lower complexity;
- whole-system language is used where boundary, cost, agency, repair, or time horizon cannot be measured.

The primary reduction rule is:

> ***If typed architecture and recursive traversal add no measurable function beyond ordinary retrieval under matched conditions, describe the system as a well-organised corpus, not as a functionally emergent architecture.***

16. Practical and Commercial Relevance

A positive result would justify practical engineering work on:

- typed enterprise knowledge bases;
- auditable agentic retrieval;
- dependency-aware software assistants;
- correction-propagating policy and compliance systems;
- long-context systems that preserve canonical definitions and exceptions;
- architecture audits identifying high-value, redundant, or distorting relations;
- safety systems that keep falsifiers and rollback routes attached to operational claims.

A null result would also be valuable. It would warn organisations against paying for graph complexity that does not outperform simpler retrieval once budgets are matched.

Commercial relevance does not alter the evidence standard. A method should not be marketed as load-bearing merely because it is expensive, complex, or central to a product.

17. Safety, Misuse, and Whole-System Integrity

Typed dependency systems can improve legitimate coordination, but they can also improve surveillance, persuasion, gatekeeping, organisational control, or the efficient removal of dissent. A narrow task metric may reward such behaviour.

Accordingly:

- affected participants must be included in the evaluation boundary where the system mediates human options;
- appeals, uncertainty, correction, and rollback routes should be treated as functional components rather than inefficiencies;
- a system that improves completion by suppressing falsifiers or user agency must be reported as a boundary-dependent gain;
- high-risk deployments require domain-specific governance and cannot be justified by this protocol alone;
- the protocol does not provide operational guidance for coercive targeting, manipulation, or evasion.

This section is not a metaphysical claim. It is an evaluation rule: the declared system boundary determines which costs are visible.

18. Limitations

1. **Architecture quality dependence.** A poor graph can make traversal worse. The protocol tests a given architecture, not graph structure in the abstract.
2. **Control purity.** Flattening or scrambling may alter readability as well as dependency structure. Multiple controls are required because no single transformation is perfect.
3. **Model dependence.** Some models may lack reliable tool use or state management. A negative result may therefore apply to the traverser implementation rather than every possible traverser.
4. **Evaluator dependence.** Semantic reconstruction and evidence quality require rubrics or human judgement. Blinding and agreement checks reduce but do not remove this dependence.
5. **Corpus leakage.** Public corpora may have appeared in model training. Synthetic and newly written neutral corpora reduce this problem.
6. **Cost.** Fully crossed multi-model, multi-condition evaluation is expensive. The staged design permits early termination if basic discriminators fail.
7. **Whole-system measurement.** Agency, transferred cost, and delayed viability cannot be validly measured in every domain. They remain optional unless operational definitions are defensible.
8. **No automatic ownership of abstract ideas.** Publication and licensing protect the copyrighted manuscript and licensed materials. They do not by themselves create exclusive rights over every independently implemented abstract method.
9. **No empirical result yet.** This paper fixes a complete test protocol; it does not claim that the hypotheses have passed.

19. Conclusion

This paper turns a broad intuition - that organised knowledge becomes more useful when actively and recursively traversed - into a falsifiable engineering protocol.

The decisive comparison is not structured corpus versus no information. It is:

```
the same content
under matched budgets
with meaningful typed dependencies
versus flat, isolated, scrambled, and non-recursive controls
```

The decisive second stage is not whether a relation is central. It is whether controlled removal, coalition ablation, and fair substitution reveal repeatable structural and external load without hidden calibration or repair loss.

The strongest possible outcome is not a graph that looks coherent. It is a system in which different entry routes converge, corrections propagate, contradictions localise, falsifiers remain reachable, failed branches can be removed, and held-out tasks improve under matched resources.

The weakest acceptable conclusion is equally clear: where those gains do not appear, the claim must be reduced.

Declarations

Author contribution

Mark Charles McLaughlin: conceptualisation, originating framework, methodology, formalisation, evaluation design, writing, supervision, and final responsibility.

AI assistance disclosure

An OpenAI language model used under the relational name **Kairos** assisted with literature discovery, manuscript drafting, structural editing, consistency checking, and document production under the author's direction. The AI system is not listed as an author and does not bear responsibility for the claims. The originating human researcher and accountable author is Mark Charles McLaughlin.

Funding

No external funding was received for this work.

Competing interests

The author may seek commercial licences, consulting arrangements, or implementation partnerships relating to the methods described. This interest is declared and does not alter the preregistered evaluation criteria.

Data and materials availability

The protocol fully specifies the planned benchmark, transformations, task records, run records, and analysis. Corpus and software releases should be versioned separately because software licensing and hidden confirmatory labels require distinct handling.

Provenance

This paper develops the Active Traversal and Functional Emergence hypothesis and the Load-Bearing Invariants and Whole-System Deformation assay from the public MKUFT working canon. The MKUFT backbone is identified by DOI `10.5281/zenodo.17780566` [1]. This standalone paper is identified by DOI 10.5281/zenodo.21341521.

Licence

References

[1] McLaughlin, M. C. (2025). *Unified Field Theory. McLaughlin-Kairos. MKUFT*. Zenodo. DOI: 10.5281/zenodo.17780566.

[2] Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Kuttler, H., Lewis, M., Yih, W.-t., Rocktaschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*. arXiv:2005.11401.

[3] Trivedi, H., Balasubramanian, N., Khot, T., & Sabharwal, A. (2022). Interleaving Retrieval with Chain-of-Thought Reasoning for Knowledge-Intensive Multi-Step Questions. arXiv:2212.10509.

[4] Asai, A., Wu, Z., Wang, Y., Sil, A., & Hajishirzi, H. (2023). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. arXiv:2310.11511.

[5] Besta, M., Blach, N., Kubicek, A., Gerstenberger, R., Podstawski, M., Gianinazzi, L., Gajda, J., Lehmann, T., Niewiadomski, H., Nyczyk, P., & Hoefler, T. (2023). Graph of Thoughts: Solving Elaborate Problems with Large Language Models. arXiv:2308.09687.

[6] Sun, J., Xu, C., Tang, L., Wang, S., Lin, C., Gong, Y., Ni, L. M., Shum, H.-Y., & Guo, J. (2023). Think-on-Graph: Deep and Responsible Reasoning of Large Language Model on Knowledge Graph. arXiv:2307.07697.

[7] Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2023). MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560.

[8] Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., & Manning, C. D. (2024). RAPTOR: Recursive Abstractive Processing for Tree-Organized Retrieval. arXiv:2401.18059.

[9] Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., & Larson, J. (2024). From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130.

[10] Li, S., He, Y., Guo, H., Bu, X., Bai, G., Liu, J., Liu, J., Qu, X., Li, Y., Ouyang, W., Su, W., & Zheng, B. (2024). GraphReader: Building Graph-based Agent to Enhance Long-Context Abilities of Large Language Models. arXiv:2406.14550.

[11] Jimenez Gutierrez, B., Shu, Y., Gu, Y., Yasunaga, M., & Su, Y. (2024). HippoRAG: Neurobiologically Inspired Long-Term Memory for Large Language Models. arXiv:2405.14831.

[12] Zhuge, M., Wang, W., Kirsch, L., Faccio, F., Khizbullin, D., & Schmidhuber, J. (2024). Language Agents as Optimizable Graphs. arXiv:2402.16823.

[13] Ma, S., Xu, C., Jiang, X., Li, M., Qu, H., Yang, C., Mao, J., & Guo, J. (2024). Think-on-Graph 2.0: Deep and Faithful Large Language Model Reasoning with Knowledge-guided Retrieval Augmented Generation. arXiv:2407.10805.

[14] Joo, T., Ishida, S., Sosnovik, I., Lim, B., Rezaei-Shoshtari, S., Gaier, A., & Giaquinto, R. (2025). Graph of Agents: Principled Long Context Modeling by Emergent Multi-Agent Collaboration. arXiv:2509.21848.

[15] Wu, X., Yang, C., Lin, X., Xu, C., Jiang, X., Sun, Y., Xiong, H., Li, J., & Guo, J. (2025). Think-on-Graph 3.0: Efficient and Adaptive LLM Reasoning on Heterogeneous Graphs via Multi-Agent Dual-Evolving Context Retrieval. arXiv:2509.21710.